NESTJS REQUEST LIFECYCLE
Повний шлях запиту + AsyncLocalStorage

Заняття №8

Node.js Pro

Лектор
Кирило Орлов

ПЛАН ЗАНЯТТЯ



- Повний цикл: шість етапів по черзі
- Guard: пускати чи ні
- Interceptor: обгортка навколо виклику
- Exception filter: остання інстанція
- AsyncLocalStorage: контекст без параметрів
- Request-id наскрізь через увесь стек

СТЕК НА СЬОГОДНІ: MIDDLEWARE · GUARD · INTERCEPTOR · PIPE · HANDLER · FILTER

Перша половина — шість етапів циклу	Друга половина — контекст запиту
middleware — сирі req/res, про контролер не знає	ALS = AsyncLocalStorage — сховище, прив’язане до async-контексту
guard — «пускати чи ні», повертає boolean	request-id — наскрізний ідентифікатор без параметрів
interceptor — обгортає виклик: до і після	чому глобальна змінна тече між запитами
pipe → handler → filter — валідація, робота, помилка	Різниця між ними — МІСЦЕ виклику, а не природа

НАСКРІЗНА НИТКА

Топ-1 питання минулого потоку: «чим middleware відрізняється від guard, interceptor і filter?». Сьогодні відповідаємо не списком визначень, а одним малюнком: усі вони — точки в одному циклі, і кожна відрізняється тим, що бачить і що може повернути.

ПРОБЛЕМА: ЧОТИРИ СУТНОСТІ, ЯКІ ЗЛИВАЮТЬСЯ В ОДНУ

«авторизація» — це guard чи middleware?

«логування» — interceptor чи middleware?

«валідація» — pipe чи guard?

«обробка помилок» — filter чи try/catch в обробнику?



Відповідь на всі чотири одна: подивись, ЩО сутність бачить і ЩО повертає — місце в циклі випливає з цього. Перевіримо на кожному паспорті.

«ПОСАДКА НА ЛІТАК»: ЦИКЛ ЯК АНАЛОГІЯ

ЧАСТИНА NEST	АНАЛОГІЯ (АЕРОПОРТ)
• Middleware • Guard • Interceptor • Pipe • Exception filter	• вхід у термінал: усім видають бирку на багаж • контроль посадкового: немає — далі не йдеш • супровідник, що йде з тобою туди й назад • митниця конкретної валізи • стійка «щось пішло не так» — звідусіль

Де аналогія ламається: в аеропорту етапи не переставиш, а interceptor у Nest вішається глобально, на контролер або на метод — порядок буде global → controller → route.

ОДИН GET — УВЕСЬ ЦИКЛ У ЛОЗІ

```
$ curl -s localhost:20800/order/42
{"id":42,"ok":true}

1 → middleware (вхід: сирі req/res, контролер ще невідомий)
2 → guard (canActivate: пускати?)
2 ← guard (true — пускаємо)
3 → interceptor (до handler)
4 → pipe ("42" → number)
4 ← pipe (віддаю 42)
5 → handler (id=42, typeof number)
5 ← handler (return об'єкт)
3 ← interceptor (після handler: бачу відповідь)
1 ← middleware (finish: відповідь уже пішла клієнту)

дужки, не черга: 1( 2 3( 4 5 )3 )1 — шари ВКЛАДЕНІ
6 → filter існує, але на happy path не з'явився ЖОДНОГО разу
на L#4 фази ми вже крутили масивом — тут кожна є ОКРЕМОЮ сутністю
```

ШІСТЬ ЕТАПІВ ЦИКЛУ — ПО ЧЕРЗІ

01. Middleware

сирі req/res, контролер ще невідомий. Тут стартує ALS. «1 ←» — це finish, після відповіді

02. Guard

boolean. false → 403, і далі НІЧОГО не виконується

03. Interceptor

обгортає handler: «3 →» до, «3 ←» після. Єдиний бачить вхід і вихід В ОДНОМУ виклику

04. Pipe

кожен аргумент окремо: «42» → 42 — трансформація й валідація

05. Handler

твій метод: id=42, typeof number. Аргументи вже готові й валідні

06. Exception filter

на happy path МОВЧИТЬ. Помилка перестрибує «5 ←» і «3 ←» — і летить сюди

GUARD: ОДНЕ ПИТАННЯ — ПУСКАТИ ЧИ НІ

```
// 02-guard.ts — guard відповідає РІВНО на одне питання
@Injectable()
export class TokenGuard implements CanActivate {
  canActivate(context: ExecutionContext): boolean {
    const req = context.switchToHttp().getRequest<Request>();
    const ok = req.headers['x-token'] === 'secret';
    console.log(ok
      ? 'guard: x-token=secret → true, пускаю далі'
      : 'guard: токена немає → false (Nest сам зробить 403…)');
    return ok;      // boolean — і ВСЕ. Відповідь збирає Nest
  }
}

@Get('secure')
@UseGuards(TokenGuard)
@UseInterceptors(ProofInterceptor)  // виконається лише після true
getSecret(): { secret: number } { return { secret: 42 }; }

// на L#4 той, хто обривав ланцюг, ПИСАВ відповідь сам: міні-Express
// не кликав next(), хук міні-Fastify робив rp.code(401).send().
// Тут інакше: у 02-guard.ts немає жодного res.status(403).

// $ curl -s -w ' [%{http_code}]' localhost:20800/secure
// {"message":"Forbidden resource","error":"Forbidden",
//  "statusCode":403} [403] — у лозі після guard БІЛЬШЕ НІЧОГО
// $ curl -s -H 'x-token: secret' …  → {"secret":42} [200]
```

Повертає boolean. Змінити відповідь не може

1 · виконується ПІСЛЯ middleware, ДО interceptor і pipe

2 · false → 403 і жоден pipe навіть не запуститься

3 · доказ обриву: після 403 у лозі — БІЛЬШЕ НІЧОГО

економія: не валідуємо тіло того, кого й так не пустимо

урок: guard — про доступ, не про дані



```
// 01-order.ts — interceptor: єдиний шар, що бачить І вхід, І вихід.
// Це не фаза, а ОБГОРТКА. На L#4 (крок 9) «до» і «після» — це ДВА
// різні хуки Fastify (preHandler і onSend), і значення від першого
// до другого доводилось класти в request. Тут — одне замикання.
import { Observable, tap } from 'rxjs';

@Injectable()
export class OrderInterceptor implements NestInterceptor {
  intercept(_context: ExecutionContext, next: CallHandler): Observable<unknown> {
    log('3 → interceptor (до handler)');            // ── ДО
    return next
      .handle()                  // ← Observable, а не готовий результат
      .pipe(tap(() => log('3 ← interceptor (після handler: бачу відповідь)')));
  }
}

// звідси «безкоштовний» timing: const t = Date.now() перед next.handle()
// видно в tap — та сама змінна, той самий виклик intercept().

// tap із success-колбеком ІГНОРУЄ помилку — вона пролітає повз і летить
// до filter-а. Саме тому в error-path лозі рядка «3 ←» НЕМАЄ.
// Треба реагувати й на помилку — додай catchError(...) у той самий pipe.

// у лозі: «3 →» до handler-а, «3 ←» після — обидва з ОДНОГО виклику
```

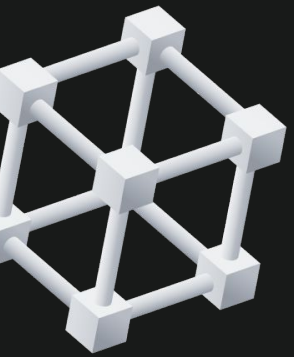
EXCEPTION FILTER: ОСТАННЯ ІНСТАНЦІЯ

глобальний (useGlobalFilters) ловить з БУДЬ-ЯКОГО шару — навіть із middleware; на контролері — лише помилки свого контролера

доменна помилка → свій код: OutOfStockError → 422 + code, sku

усе інше → 500 БЕЗ стеку й тексту помилки назовні

у лог — повний стек і requestId; клієнту — код, коротке повідомлення й той самий requestId для підтримки



```
GET /order/0 — handler кидає

1 → middleware (вхід: …)
2 → guard (canActivate: пускати?)
2 ← guard (true — пускаємо)
3 → interceptor (до handler)
4 → pipe ("0" → number)
4 ← pipe (віддаю 0)
5 → handler (id=0, typeof number)
6 → filter (спіймав
    "handler впав" → 500)
1 ← middleware (finish: …)

«5 ← handler» і «3 ← interceptor»
у лозі НЕМАЄ — помилка
перестрибнула обидва виходи
```



```
// 03-filter.ts — сервіс кидає ДОМЕННІ помилки й нічого не знає про HTTP
export class OutOfStockError extends Error {
  constructor(public readonly sku: string) {
    super(`товару «${sku}» немає на складі`);
    this.name = 'OutOfStockError';
  }
}
// добір за ТИПОМ у @Catch(...) — не арність 4 (Express) і не if усередині
// setErrorHandler (Fastify). Фільтрів кілька — Nest обере найвужчий.
@Catch()                          // без аргументів = остання лінія, ловить УСЕ
export class DomainExceptionFilter implements ExceptionFilter {
  catch(exception: Error, host: ArgumentsHost): void {
    const res = host.switchToHttp().getResponse<Response>();
    if (exception instanceof OutOfStockError) {
      res.status(422).json({ statusCode: 422, code: 'OUT_OF_STOCK',
        message: exception.message, sku: exception.sku });
      return;
    }                             // (+ гілка HttpException — не чіпаємо)
    console.log(exception.stack);   // повний стек — ЛИШЕ в серверний лог
    res.status(500).json({ statusCode: 500, message: 'Internal server error' });
  }
}
// різалки схеми, як на L#4, у Nest немає: що поклав у json — те й піде
// GET /buy/rare → 422 · GET /buy/boom → 500 без 'boom' і без стеку
```

ХТО ЩО БАЧИТЬ І ЩО ПОВЕРТАЄ

Сутність	Бачить	Повертає
Middleware	сирі req / res	нічого — кличе next() або обриває
Guard	запит + метадані маршруту	boolean — і тільки
Interceptor	виклик цілком: до і після	Observable — може підмінити відповідь
Pipe	один аргумент + його метадані	значення або кидає помилку
Filter	помилку з будь-якого етапу	HTTP-відповідь замість падіння

ПОРЯДОК: ГЛОБАЛЬНИЙ → КОНТРОЛЕР → МАРШРУТ

на кожному етапі спершу глобальні, потім контролера, потім методу

guard-и йдуть по черзі — перший false зупиняє ланцюг

interceptor'и вкладаються: global обгортає controller, той — route

«після» в interceptor йде у ЗВОРОТНОМУ порядку — як стек

filter шукається знизу вгору: route → controller → global

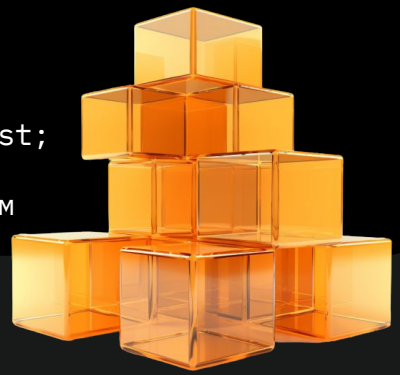
middleware — єдиний, хто налаштовується в модулі, а не декоратором

РІВНІ ВКЛАДАЮТЬСЯ, ЯК ЦИБУЛИНА

```
global      →  «до»
  controller   →  «до»
    route         →  «до»
          handler
    route         ←  «після»
  controller   ←  «після»
global      ←  «після»

«до» — вниз по рівнях,
«після» — назад, як стек

рівні — з документації Nest;
порядок ЕТАПІВ ми довели
логом і зафіксували тестом
```



```
// 05-order.spec.ts — порядок не «на словах»: toEqual по масиву логів
it('happy path: middleware → guard → interceptor → pipe → handler …', async () => {
  const res = await request(app.getHttpServer()).get('/order/42').expect(200);
  expect(res.body).toEqual({ id: 42, ok: true });
  await waitForLog(10);            // finish прилітає вже ПІСЛЯ відповіді
  expect(lifecycleLog).toEqual([
    '1 → middleware (вхід: сирі req/res, контролер ще невідомий)',
    '2 → guard (canActivate: пускати?)',
    '2 ← guard (true — пускаємо)',
    '3 → interceptor (до handler)',
    '4 → pipe ("42" → number)',
    '4 ← pipe (віддаю 42)',
    '5 → handler (id=42, typeof number)',
    '5 ← handler (return об’єкт)',
    '3 ← interceptor (після handler: бачу відповідь)',
    '1 ← middleware (finish: відповідь уже пішла клієнту)',
  ]);
});
it('handler кидає: «після»-гілки interceptor-а НЕМАЄ, замість неї — filter', async () => {
  await request(app.getHttpServer()).get('/order/0').expect(500);
  await waitForLog(9);
  expect(lifecycleLog).not.toContain('3 ← interceptor (після handler: бачу відповідь)');
});
// npm test → Test Suites: 1 passed, 1 total · Tests: 2 passed, 2 total
```

«ГАРДЕРОБНИЙ НОМЕРОК»: ASYNCLOCALSTORAGE

ЧАСТИНА NODE

- als.run(store, fn)
- als.getStore()
- глобальна змінна
- паралельні запити
- requestId у сховищі

АНАЛОГІЯ (ГАРДЕРОБ)

- видали номерок на вході
- показав номерок — отримав свої речі
- одна полиця на всіх — речі плутаються
- десять гостей — жоден не переплутав
- номер не треба нести через усі зали

ЧОМУ НЕ ГЛОБАЛЬНА ЗМІННА

let currentRequestId

поки перший запит чекає на await, другий її перезаписує

передавати параметром

працює, але requestId тягнеться крізь усі сигнатури

AsyncLocalStorage

окреме сховище на кожен async-контекст. Нуль параметрів

Це вже було на L#2

Там ми бачили той самий баг у логері: під навантаженням у логах одного запиту з'являвся requestId іншого. Причина не в Node і не в «багатопотоковості» — Node однопотоковий. Причина в тому, що await повертає керування event loop, і той встигає почати наступний запит. Одна глобальна змінна на всіх — означає один requestId на всі паралельні запити. ALS дає кожному запиту свій.

```
// 04-als.ts — ОДИН інстанс на процес, ОКРЕМИЙ store на кожен запит
const als = new AsyncLocalStorage<{ requestId: string }>();
export class RequestContext {
  static get requestId(): string {
    return als.getStore()?.requestId ?? 'поза-контекстом';
  }
}
// наївна альтернатива: поле singleton-провайдера (L#4, крок 8 — час життя)
export class NaiveContext { static requestId = 'поза-контекстом'; }

// middleware — найраніший шар: лише звідси можна обгорнути ВЕСЬ ланцюг
export function contextMiddleware(req: Request, res: Response, next: NextFunction): void {
  const requestId = (req.headers['x-request-id'] as string | undefined) ?? randomUUID();
  res.setHeader('x-request-id', requestId);  // клієнт отримує id назад
  NaiveContext.requestId = requestId;        // ← «а що як просто у поле?»
  als.run({ requestId }, () => next());      // усе після next() бачить цей store
}

// глибина 3 (ArchiveService): жодного requestId у сигнатурі
async load(): Promise<string[]> {
  await sleep(200);  // саме тут event loop перемикається на інший запит
  ctxLog('archive: глибина 3 — id дістав зі сховища, у сигнатурі його немає');
  return ['рядок-1', 'рядок-2'];
}
```



REQUEST-ID: НАСКРІЗНИЙ ІДЕНТИФІКАТОР

01. Беремо з X-Request-Id або генеруємо самі

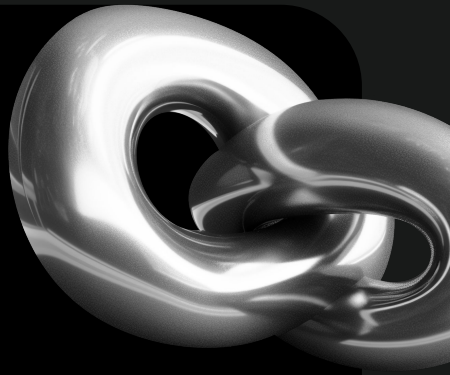
02. Кладемо в ALS у middleware — найранішій точці

03. Логер підмішує його в КОЖЕН рядок автоматично

04. Повертаємо клієнту заголовком — щоб він назвав його в підтримці

05. Передаємо далі по HTTP у сусідні сервіси — так збирається трасування

```
$ curl -H 'x-request-id: A' localhost:20800/report &  curl -H 'x-request-id: B' localhost:20800/report &  wait
лог: [als=A][поле=A] handler → [als=A][поле=A] report → [als=B][поле=B] handler → [als=B][поле=B] report →
     [als=A][поле=B] archive: глибина 3 → [als=A][поле=B] report: готово → [als=B][поле=B] archive → готово
відповіді: A → {"requestId":"A","naiveRequestId":"B","rows":2} · B → {"requestId":"B",…} — als НЕ переплутано, поле — так
$ curl -si -H 'x-request-id: my-own-id' localhost:20800/report | grep -i x-request-id  →  x-request-id: my-own-id
$ curl -si localhost:20800/report | grep -i x-request-id  →  x-request-id: 0fab62b3-8f52-48ba-ab94-85cc5dd79426
```



```
// 01-order.ts — де ЩО вішається (усі шість шарів разом)
@Controller()
@UseFilters(OrderFilter)                      // filter — на контролер
export class OrderController {
  @Get('order/:id')
  @UseGuards(OrderGuard)                      // guard — на маршрут
  @UseInterceptors(OrderInterceptor)          // interceptor — на маршрут
  getOrder(@Param('id', OrderPipe) id: number): { id: number; ok: boolean } {
    …                                         // pipe — на КОНКРЕТНИЙ аргумент
  }
}

@Module({ controllers: [OrderController] })
export class AppModule implements NestModule {
  configure(consumer: MiddlewareConsumer): void {
    consumer.apply(orderMiddleware).forRoutes(OrderController);
  }                                           // middleware — ЛИШЕ через модуль
}

// на весь застосунок: providers: [{ provide: APP_GUARD /
//   APP_INTERCEPTOR / APP_FILTER, useClass: … }] (дока Nest) —
//   app.enableShutdownHooks() перед listen() — є в усіх 4 кроках
```

ПАСТКИ, НА ЯКИХ ЗАСТРЯГАЮТЬ

ALS ініціалізували не в middleware → глибокі виклики бачать undefined

als.run обгортає не весь обробник → контекст губиться після await

REQUEST-scope замість ALS → весь граф перебудовується на кожен запит

filter не глобальний → частина помилок летить повз нього

стек у відповіді клієнту → витік внутрішньої структури

забули enableShutdownHooks → пули БД не закриваються при SIGTERM

ЯКУ ЗАДАЧУ ЯКОЮ СУТНІСТЮ ВИРІШУВАТИ

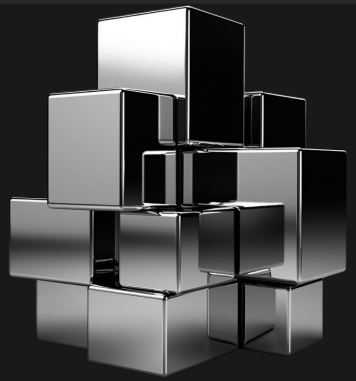
Задача	Правильна сутність	Чому саме вона
Авторизація	Guard	потрібен лише boolean і метадані маршруту
Логування часу	Interceptor	треба бачити і початок, і кінець виклику
Валідація тіла	Pipe	працює з одним аргументом і його типом
Request-id	Middleware + ALS	потрібен найраніший етап і сирий res для заголовка
Формат помилок	Exception filter	єдина точка, куди сходяться всі помилки

ЩО СВІДОМО ЛИШАЄМО НА ПОТІМ

OpenTelemetry і структуроване логування через pino — Лекція 29: до нашого requestId додасться traceparent.

Helmet, CORS, rate-limit — свідомо не чіпаємо, це Лекція 27. Технічно це middleware або guard, і після сьогоднішнього ти вже знаєш, куди їх ставити.

⚠ REQUEST-scope провайдери навмисно не рекомендуємо: вони перебудовують граф на кожен запит, а ALS дає той самий ефект без цієї ціни.



ПІДСУМКИ + МІСТ ДО L#9

ЦИКЛ

middleware → guard → interceptor → pipe → handler — і назад як стек; filter лише при помилці

КОНТЕКСТ

AsyncLocalStorage дає requestId скрізь без жодного параметра

далі

L#9 — дизайн API: версіонування, пагінація, формат помилок

ДЗ #8

8 балів · фінал mini-Nest: Guard, Interceptor, Zod-pipe, Exception filter, request-id через AsyncLocalStorage. Тест на ПОРЯДОК етапів обов’язковий. Частина 3 з 3

РІЗНИЦЯ

не в природі, а в тому, що сутність бачить і що повертає